

1. API server

- User queries received through `kubectl` are routed via this node, making it the cluster's primary gateway.
- It controls who can access resources and make requests by acting as a gatekeeper for authenticating and verifying requests.

2. Scheduler

- Determines on which node the next pod will be spun up. However, this action does NOT spin up the pod itself; Kubelet is responsible for this step.

3. Controller manager

- Identifies when the cluster's status has changed (for example, when pods have died) and attempts to return it to its initial, created state.
- In case the launch of a pod is unsuccessful for whatever reason, the Controller Manager will request that the Scheduler choose a node for the launch of a new pod. After that, Kubelet will begin the spinning of the new pod.

4. etcd

- This is the key-value storage used by the cluster.
- It is the location where any adjustments that have been made to the cluster are recorded.
- This location is used only for the storage of information pertaining to the cluster itself; no application data is ever stored there. It is important to remember that the master node is not a worker but rather the command and control hub for the cluster. The status information is saved in etcd so that the previously described procedures may acquire knowledge of the cluster.

Slave/worker node(s)

In order for Kubernetes to be able to connect with the nodes and launch pods

on its own, the Central Concentration of Servants and Employees (s) component of Kubernetes mandates that three different node processes be installed on each and every worker node. The following three steps are required.

1. Kubelet a.k.a. *kubelet*

- Facilitates two-way communication between the node and the container.
- It is responsible for reading configuration files and starting the pod by using the container runtime that is provided by the node 2 server (more on this later!). It's the OS for containers.

2. Container runtime

- A computer setup that already has a container runtime installed (e.g., Docker, containerd)
- Kube's proxy, often known as kube-proxy, comes in third.

3. Kube Proxy a.k.a. *Kube-proxy*

- It is located between computers and intelligently forwards requests, either inside a node or to other nodes.
- Since we now have an understanding of how K8s operates, we are able to investigate the Kubernetes building pieces that are most often seen throughout the process of application deployment.

Components of Kubernetes

Now that we understand how K8s functions, we can examine the Kubernetes building blocks most often seen during application deployment.

1. Pod

- The smallest building element that makes up a Pod K8, and the location where a real instance of your software is deployed.
- Over the top of a receptacle is an abstraction.
- Every pod has its own one-of-a-kind IP address (public or private).
- Until the pod service is rebuilt, the IP address will be changed.

2. Service

- Despite the ephemeral nature of pods, the service provides a means to 'give' them a permanent IP address.
- Even if the pod is destroyed and then re-created while it is still connected to the service, it will keep the same IP address.
- It redirects traffic to pods while maintaining a single IP address, functioning in a manner that is similar to that of a load balancer.
- Similar to load balancers, the service may either be internal or external. An external service is one that is exposed to the general public (has public IP), while an internal service is one that is intended for use with internal applications (has private IP).

3. Ingress

- Because of services, we may now have a web service that is listening on a certain port (for example, 8080) at an IP address (for example, 10.104.35). Unfortunately, it is not possible to browse a public-facing application by using the address `http://10.104.35:8080`. This is because it is not a practical address.
- This indicates that we are in need of a domain-based entry point (such as `https://my-domain-name.com`) that directs traffic to the service (such as `http://10.104.35:8080`).
- In essence, Ingress makes available to internal cluster services both HTTP and HTTPs routes that originate from the outside.
- Due to SSL termination, traffic destined for the service and its pods will not be encrypted (also known as SSL offloading).
- Just so there is no confusion, constructing an Ingress resource serves no use on its own. An Ingress Controller is required in addition to an Ingress in order for the latter to be finished.